SEMINAR · THURSDAY 29 & FRIDAY 30 OCTOBER 2026

From Code to Production

Cloud deployment in practice

code ▷ container ▷ cloud ▷ automation ▷ operations

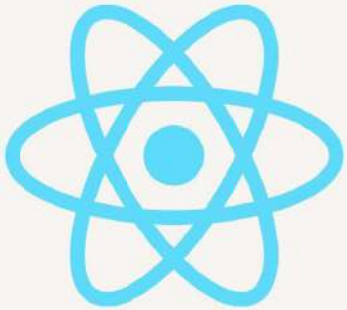
David Cediel

About Me

Software Engineer at Sezzle

EMSE Graduated

Current Stack: AWS (EKS)







THE GOAL

By Friday, your API runs on a public URL in the cloud, and you know every step that got it there.

A small app on purpose: the focus is how it gets deployed, not the code.

Agenda

Thursday 29 October

Welcome

1. The deployment landscape

2. Containers (Hands-on 1)

3a. Running services in the cloud: theory

Friday 30 October

Recap

3b. Running services in the cloud: practice (Hands-on 2)

4. Automation: CI/CD and infrastructure as code

5. Operating in production

Wrap-up and assignment

How every block works

I explain → **I show it live** → **You do it** → **Quick quiz**

15% · ATTENDANCE

Be here both days. A short form appears during each session.

85% · ASSIGNMENT

Deploy your own app and explain it in a video. Details on Friday

SETUP CHECK · 4 MIN

Is your machine ready?

1 · THE REPO

Cloned, on branch **step-0-hello-world**

2 · DOCKER

Running, or a Codespace open

3 · AZURE

Azure for Students active (needed tomorrow)

```
# start it now, it downloads in the background (874 MB)
docker pull golang:1.26
```

BLOCK 1

The deployment landscape

Goal: a shared map, so every later block has a place on it.

- How we got here
- Service models
- Who is responsible
- Where our app fits

Four ways to deploy

SINCE THE 1990s



Bare metal

1 Dev builds app.jar

2 Sends it to the ops team with an install guide

3 Ops copies it to the server, installs Java by hand, restarts at night

PROBLEM

Weeks to get a server, mostly idle

SINCE THE 2000s



Virtual machines

1 Dev builds app.jar

2 Ops creates a VM from a template in minutes

3 A script installs Java and the app on it

SOLVED

Better use, minutes to create

NEW PROBLEM

Each one carries a full OS

SINCE 2013 · TODAY'S DEFAULT



Containers

1 CI builds an image: app, Java and libraries

2 Pushes it to a registry

3 The server pulls and runs that exact image

SOLVED

Megabytes, seconds, runs the same anywhere

NEW PROBLEM

Hundreds to manage: orchestration

SINCE 2014



Serverless

1 Dev pushes code or an image

2 The provider builds it and runs it

3 No server to manage at all

SOLVED

No servers, pay per use, scale to zero

NEW PROBLEM

Cold starts, lock-in

All four are still in use. Each one moves more work from people to automation.

Service models, as a pizza place 1 of 2

ON-PREMISES

Your own data center

Your own restaurant

You own the building, the ovens, everything.

- App and data
- Runtime
- Operating system
- Virtualization
- Hardware

IAAS

Infrastructure as a Service

Rent a kitchen

Ovens included. You bring cooks, recipes, ingredients. *Azure VMs*

- App and data
- Runtime
- Operating system
- Virtualization
- Hardware

CAAS · WHAT WE USE

Containers as a Service

Bring a ready pizza

Packed by you (the container). They bake and serve as many as needed. *Container Apps*

- App and data
- Runtime (in the image)
- Operating system
- Virtualization
- Hardware

you manage | the provider manages

Service models, as a pizza place 2 of 2

PAAS

Platform as a Service

Hand over the recipe

You give the recipe (your code). They cook and serve. *App Service*

- **App and data**
- Runtime
- Operating system
- Virtualization
- Hardware

FAAS · "SERVERLESS"

Functions as a Service

Pizza by the slice

One slice cooked per order. No orders, no cost. *Azure Functions*

- **Functions and data**
- Runtime
- Operating system
- Virtualization
- Hardware

SAAS

Software as a Service

Eat at a restaurant

You just eat. *Jira, Slack*

- App (you bring only data)
- Runtime
- Operating system
- Virtualization
- Hardware

To the right: **you do less, and you control less.**

The same, layer by layer

	On-prem your servers	**IaaS** Virtual Machines	**CaaS** Container Apps, AKS	**PaaS** App Service	**FaaS** Functions
App code and data	You	You	You	You	You
Runtime	You	You	You (image)	Provider	Provider
Operating system	You	You	Provider	Provider	Provider
Virtualization	You	Provider	Provider	Provider	Provider
Hardware, network	You	Provider	Provider	Provider	Provider

Left to right: **less control, less ops work, more lock-in.**

Shared responsibility

THE PROVIDER, ALWAYS

Datacenters, hardware, physical network

DEPENDS ON THE MODEL

OS patches, runtime updates, network rules, scaling

IaaS: you patch the OS. PaaS: the provider does.

YOU, ALWAYS

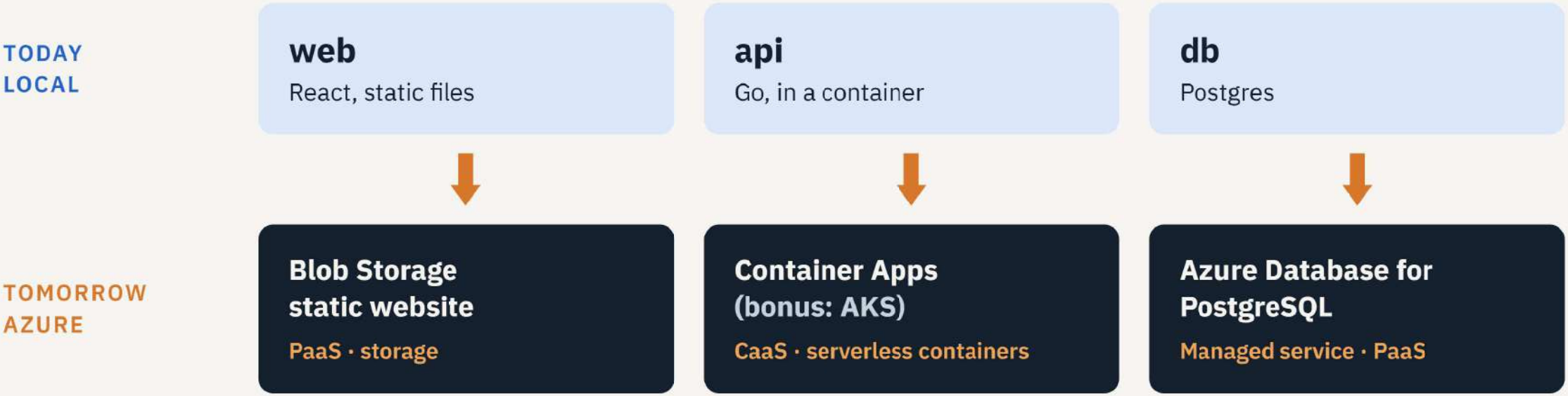
Your data, your code, who has access

⚠ Most cloud incidents are on **your** side: public storage, leaked keys, permissions that are too broad.

EXAMPLE · CAPITAL ONE, 2019

A misconfigured firewall and a role with too many permissions let an attacker copy data of about 100 million customers from the bank's cloud storage. AWS worked as designed: the configuration was the bank's.

Our app, and where it ends up



Deliberately tiny: a notes app. **The deployment is the point.**

QUIZ · BLOCK 1 · 1 OF 2

Who patches the operating system in a PaaS?

- **A** You
- **B** The cloud provider
- **C** Nobody, it isn’t needed
- **D** Your CI pipeline

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 1 · 1 OF 2

Who patches the operating system in a PaaS?

A You

B The cloud provider ✓

C Nobody, it isn't needed

D Your CI pipeline

Why: In PaaS the provider runs and patches everything below your code. You still update your app's own libraries.

QUIZ · BLOCK 1 · 2 OF 2

Which model can scale to zero when nobody uses it?

- **A** On-premises
- **B** IaaS (a VM)
- **C** FaaS / serverless
- **D** All of them

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 1 · 2 OF 2

Which model can scale to zero when nobody uses it?

A On-premises

B IaaS (a VM)

C **FaaS / serverless** ✓

D All of them

Why: Functions only run when an event arrives. No requests: nothing running, nothing to pay. Container Apps can do this too.

BLOCK 2

Containers

Goal: you containerize the API yourself.

Theory	**Demo: bad vs good**	**Hands-on 1**	**Many containers: Compose**

“But it works on my machine…”

A container image packs your app with everything it needs, so it runs the same on your laptop, in CI and in the cloud.

What is Docker?



A tool to **build, share and run containers**.

Created by Solomon Hykes at dotCloud, open source since 2013.

Written in Go. 

Not the inventor of containers: Linux had them already. Docker made them easy: Dockerfile, images, Docker Hub.

Now an open standard (OCI): the same images run with Podman, containerd, Kubernetes.

THE PROBLEM IT SOLVES

Shipping software was fragile:

Different versions on every machine

Long install guides for ops

“Works on my machine”

Apps on one server breaking each other

Like shipping containers: **one standard box**, and any ship can carry it.

Image vs container

LIKE A CLASS

Image

Read-only template: app, dependencies and filesystem. Built from a Dockerfile.

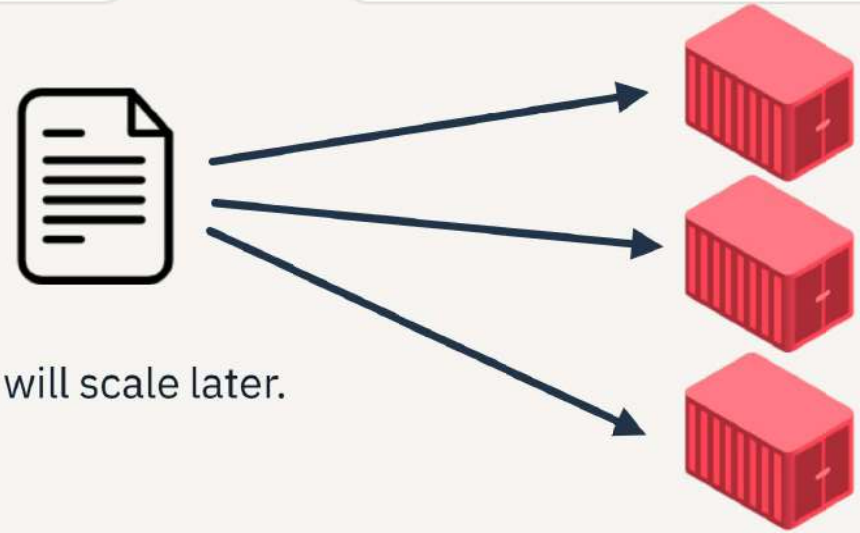
```
docker build
```

LIKE AN OBJECT

Container

A running instance of an image. Ephemeral: its writable layer is lost when it is deleted.

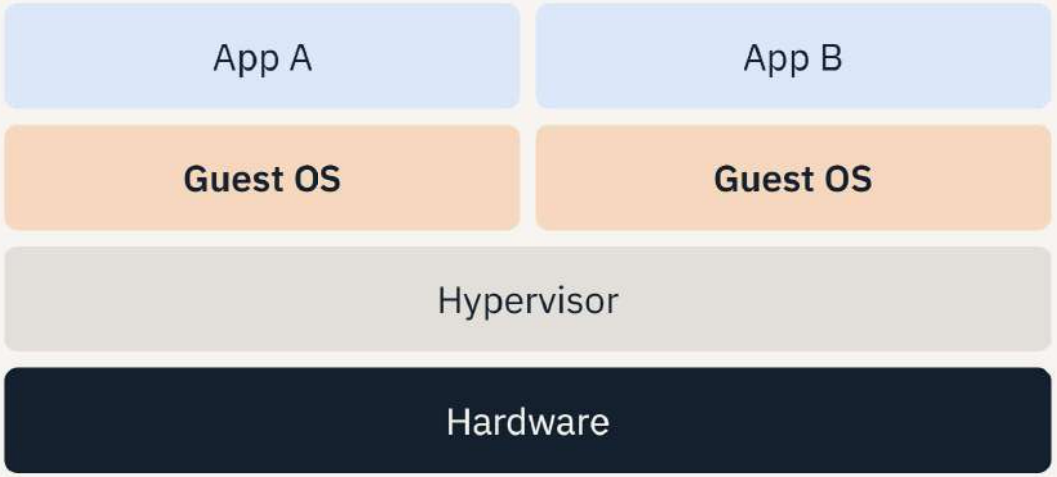
```
docker run
```



One image, many containers: that is how we will scale later.

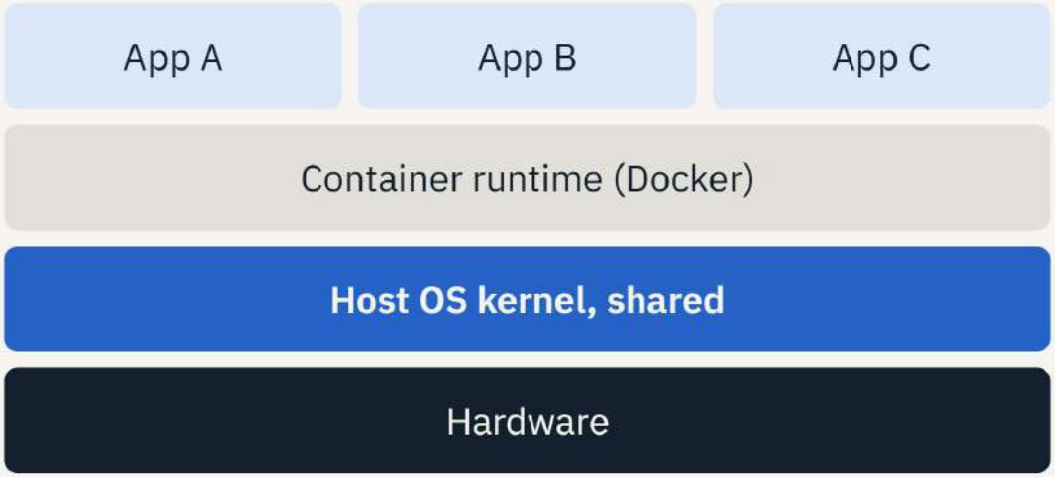
Containers vs virtual machines

Virtual machines



Each VM boots a **full OS**: gigabytes, minutes to start.

Containers



Containers are isolated processes: **namespaces** limit what they see, **cgroups** limit how much CPU and memory they use.

Layers and the build cache

`FROM golang:1.26 AS build`	cached
`WORKDIR /src`	cached
`COPY go.mod ./`	cached
`RUN go mod download`	cached
`COPY . .`	you changed main.go
`RUN go build …`	rebuilt

A change rebuilds its layer and every layer after it.

So: what rarely changes first (base image, dependencies), your code last.

Edit code → seconds. Edit dependencies → full download.

Our API’s Dockerfile

```
# ---- Stage 1: build ----
FROM golang:1.26 AS build
WORKDIR /src
COPY go.mod ./ # deps first: cached
RUN go mod download
COPY . . # code changes often: last
RUN CGO_ENABLED=0 go build -o /bin/api .
# ---- Stage 2: run ----
FROM gcr.io/distroless/static-debian13:nonroot
COPY --from=build /bin/api /api
EXPOSE 8080 # documentation only
ENTRYPOINT ["/api"]
```

FROM base image

WORKDIR folder for the next steps

COPY files into the image

RUN a command at **build** time

EXPOSE the port the app uses

ENTRYPOINT / CMD what runs at **run** time

ENV, USER env vars, non-root user

Multi-stage builds: ship only the result

STAGE 1 · BUILD

`golang:1.26`

Go compiler, source code, module cache

874 MB



`/bin/api`

STAGE 2 · RUN (WHAT YOU SHIP)

`distroless/static:nonroot`

Only the binary. No compiler, no shell, no root.

11.2 MB

About 80× smaller: faster pulls and deploys, and less inside for an attacker to use.

Registries: where images live



`docker.io/davidcediel12/seminar-api:3.1`

registry user repository tag = version









The 12-factor essentials

Config in env vars

URLs and passwords come from outside. Same image for dev, test and prod.

Stateless processes

State lives in a database or cache. Any copy can be killed or replaced.

Logs to stdout

Don’t manage log files. The platform collects the output.

Also worth knowing: fast start and graceful shutdown, dev/prod parity, build → release → run.

LIVE DEMO · 8 MIN

A bad Dockerfile vs a good one

Bad	Good
FROM ubuntu, then install everything	Official, slim or distroless base image
COPY . . before installing dependencies	Dependencies first, code last
ENV DB_PASSWORD=secret123	Secrets injected at run time
Runs as root	Non-root user
Build tools in the final image	Multi-stage build
:latest everywhere	Pinned versions

Commands you will use today

```
# build and run
docker build -t api ./api
docker run --rm -p 8081:8080 api
docker run -e APP_MESSAGE=Hi api
```

```
# look inside
docker ps
docker logs <container>
docker images
```

```
# share (Hands-on 2)
docker login
docker tag api <user>/seminar-api:1.0
docker push <user>/seminar-api:1.0
```

```
# many containers (Compose)
docker compose up --build
docker compose down
docker compose down -v
```

HANDS-ON 1 · 15 MIN branch **step-0-hello-world**

Containerize the API

1 · Write it

7 min

Open **api/Dockerfile** and replace each **TODO** with one instruction. Read the “why”.

2 · Build and run

4 min

```
docker build -t api ./api
docker run --rm -p 8081:8080 api
```

Open **localhost:8081/api/message**
You see: **“Hello from the API”**

3 · A second one

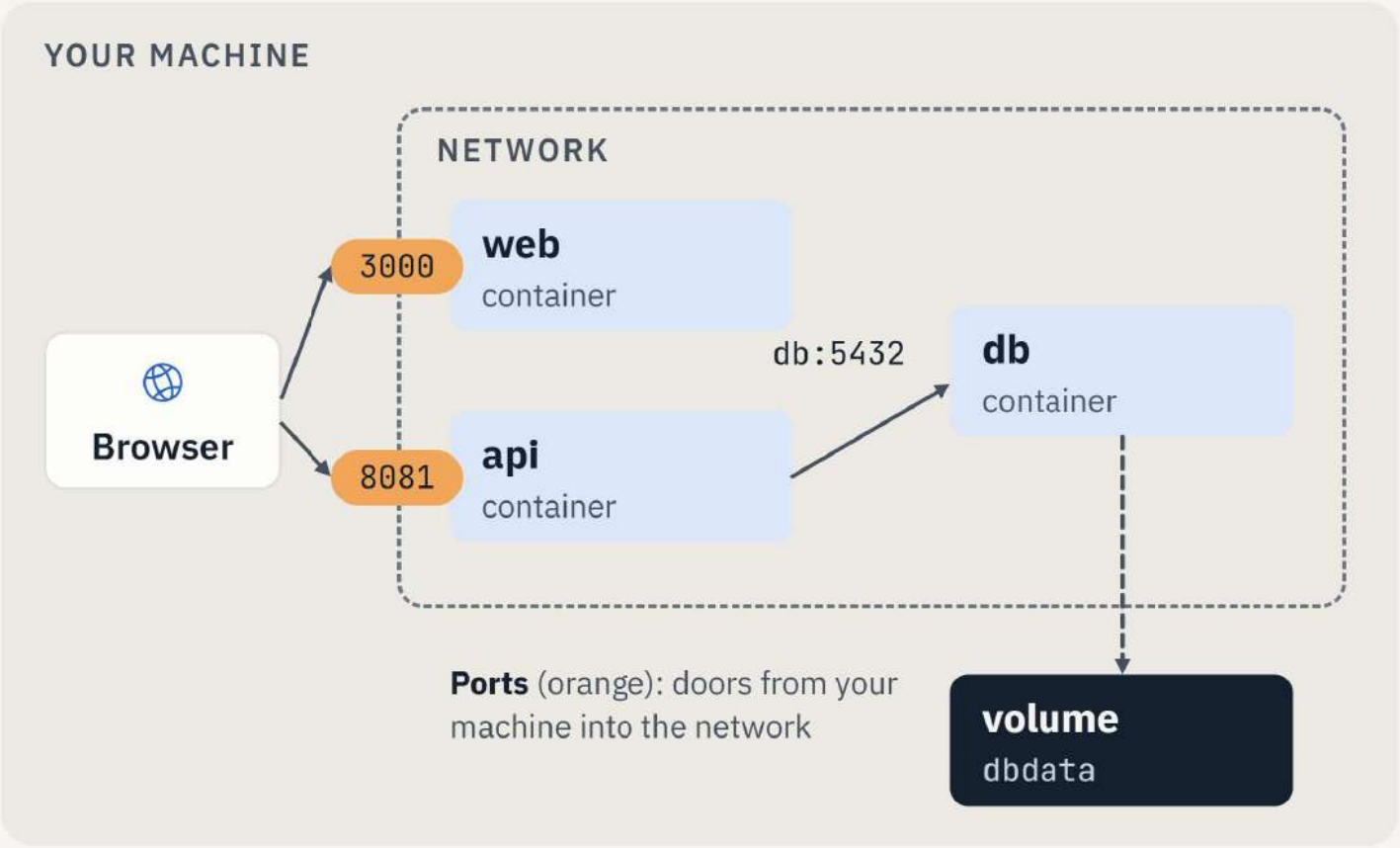
4 min · in a new terminal

```
docker run --rm -p 8082:8080 \
-e APP_MESSAGE="Hi from me" api
```

:8082 says “Hi from me”
:8081 still says “Hello…”
Same image, two configs, **no rebuild**

Behind? git stash then git switch step-1-containerized and build again.

A real app is many containers



Doing this by hand:

Who starts the db before the api?

Who restarts a container that crashes?

How do you update one without breaking the rest?

Does every teammate set it all up again?

What an orchestrator does

You **describe the desired state** in a file. The tool makes it real, and keeps it that way.

Start and connect
in the right order, on one network

Heal
restart what crashes

Scale
run more copies under load

Update
new versions without downtime

TODAY · ONE MACHINE · DEVELOPMENT

Docker Compose

NEXT · MANY MACHINES · PRODUCTION

Kubernetes, Container Apps

Anatomy of compose.yaml

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: dev_password
    volumes:
      - dbdata:/var/lib/postgresql/data
  api:
    build: ./api
    ports:
      - "8081:8080"
    environment:
      DB_URL: postgres://…@db:5432/notes
    depends_on: [db]
  web:
    build: ./web
    ports: ["3000:8080"]
volumes:
  dbdata:
```

services one entry per container

image / build pull an image, or build a Dockerfile

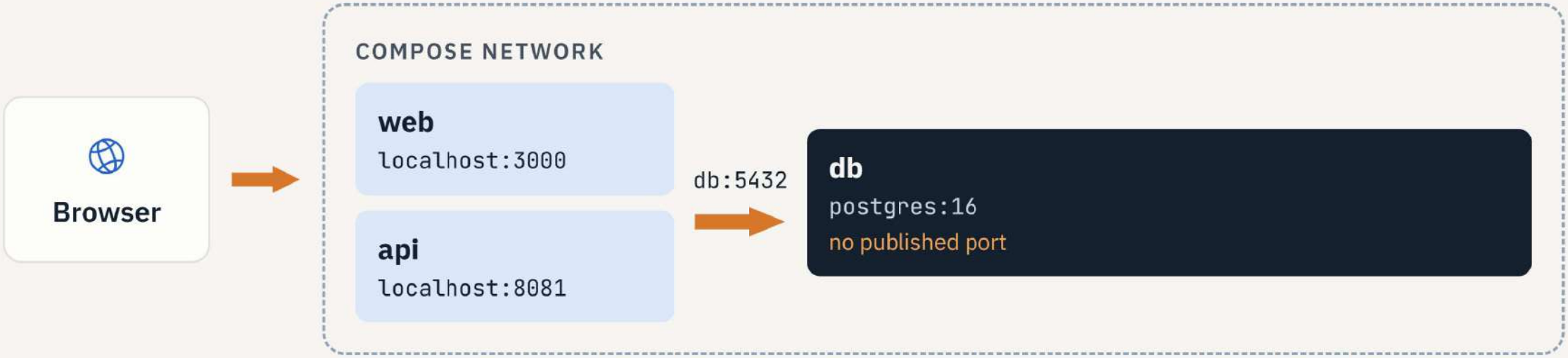
ports host:container, reachable from your machine

environment config as env vars

depends_on start order

volumes data that outlives the container

Networking: service name = hostname



With ports: reachable from your machine (web, api).

Without ports: only other services can reach it (db).

The API finds the DB at db:5432, by its service name.

Volumes: data that outlives the container

DB CONTAINER

Writable layer: **deleted with the container**

`/var/lib/postgresql/data`

mounted



VOLUME

`dbdata`

Managed by Docker, outside the container. **Kept until you delete it.**

```
- dbdata:/var/lib/postgresql/data
```

volume name — folder inside the container

LIVE DEMO · 4 MIN branch **step-2-compose-db**

The whole app, one command

```
git switch step-2-compose-db
docker compose up --build
# open http://localhost:3000, add a note
docker compose ps
docker compose logs api
```

Three containers from one file

The API waits for the database to be ready

Logs of every service in one place

LIVE DEMO · CONTINUED

Where does the note go?

```
docker compose down
docker compose up
```

The note is still there

Containers were deleted, the volume was kept.

```
docker compose down -v
docker compose up
```

The note is gone

-v deleted the volume too.

Containers are ephemeral. State belongs in backing services.

QUIZ · BLOCK 2 · 1 OF 3

Why does the order of Dockerfile instructions affect build time?

A Docker runs them in parallel

B Later instructions are always slower

C It doesn't, order never matters

D A change rebuilds that layer and every layer after it

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 2 · 1 OF 3

Why does the order of Dockerfile instructions affect build time?

A Docker runs them in parallel

B Later instructions are always slower

C It doesn't, order never matters

D A change rebuilds that layer and every layer after it ✓

Why: Docker reuses cached layers until one changes, then rebuilds everything after it. Dependencies first, code last.

QUIZ · BLOCK 2 · 2 OF 3

Where should a database password go?

A In the Dockerfile, with ENV

B In the code, as a constant

C Injected at run time: env var or secret

D In a committed .env file

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 2 · 2 OF 3

Where should a database password go?

A In the Dockerfile, with ENV

B In the code, as a constant

C Injected at run time: env var or secret ✓

D In a committed .env file

Why: The image stays the same everywhere; the password is given when it runs. Anyone who can pull an image can read its ENV values.

QUIZ · BLOCK 2 · 3 OF 3

How does the API find the database in Compose?

A By its service name, db

B By its IP address

C Through localhost

D Through a port published on your machine

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 2 · 3 OF 3

How does the API find the database in Compose?

A By its service name, db ✓

B By its IP address

C Through localhost

D Through a port published on your machine

Why: Compose gives every service a DNS name on its network. The API connects to db:5432: IPs change, names don’t.

BLOCK 3A

Running services in the cloud

Goal: understand how the cloud runs containers, and what Kubernetes is.

- Laptop vs cloud
- What Kubernetes is
- Its building blocks
- Do you need it?

From your laptop to the cloud: what changes

On your laptop (Compose)	In the cloud
Services find each other by name	The same idea, across many machines
Database in a container, with a volume	A managed database: backups, failover, patches
localhost:8081	A public URL with HTTPS, behind a load balancer
Env vars in compose.yaml	Env vars and secrets set in the platform
One copy of each service	Several copies, autoscaling, restarts on failure
Free	Costs money: delete what you don’t use

What is Kubernetes?

An **open-source container orchestrator** for many machines.

Created by Google in 2014, from its internal system Borg. **Written in Go.** 

Owned by no single company: given to the CNCF (Linux Foundation) in 2015.

Runs anywhere: AKS, EKS, GKE, or your own servers.

Often written **K8s**: K, 8 letters, s.

THE PROBLEM IT SOLVES

Hundreds of containers on dozens of machines:

Which machine runs each one?

Who restarts it when it crashes?

How do we add copies under load?

How do we update without downtime?

How do they find each other?

Compose answered this for **one machine**. Kubernetes answers it for **a whole fleet**.

Kubernetes: you declare, it keeps it true

1 · YOU DECLARE

"I want **3 copies** of seminar-api:3.1"

→

2 · IT COMPARES

Wanted 3, running **2** (one crashed)

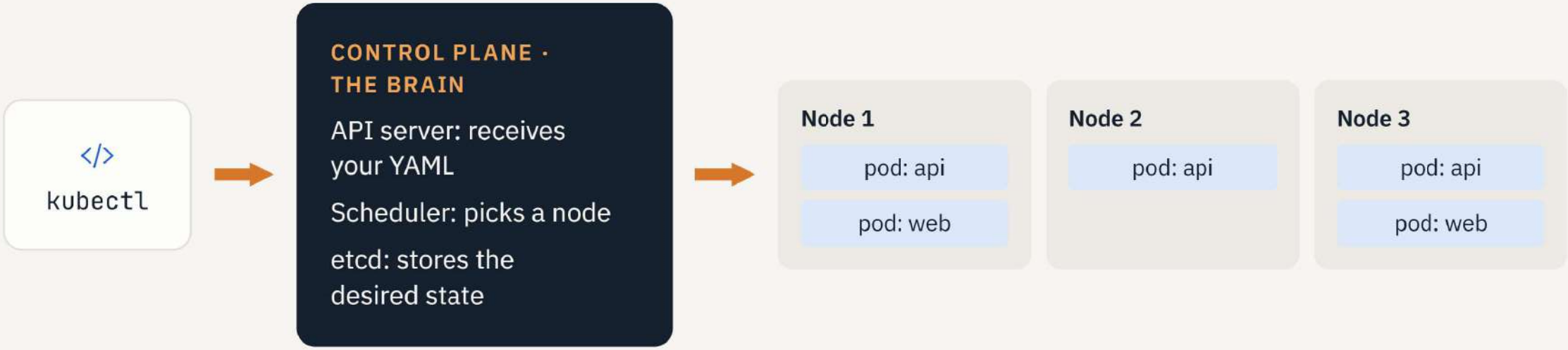
→

3 · IT ACTS

Starts **1 new copy**

Then it checks again, and again, forever. **That loop is why a crashed container comes back on its own.**

Inside a cluster



Node = a machine (VM) that runs containers.

Pod = the smallest unit: one container, with its own IP. Disposable.

Managed (AKS, EKS) = the provider runs the brain for you.

Pod

Building blocks 1 of 5

The smallest thing Kubernetes runs: **usually one container**.

It has its own IP address.

Disposable: replaced, never repaired. A new pod gets a new IP.

You almost never create one by hand.

```
# the pod part of our Deployment
containers:
  - name: api
    image: seminar-api:3.1
    ports: [{ containerPort: 8080 }]
```



Pod vs container

Building blocks 1 of 5 · a closer look

	Container	Pod
What	A running image	A wrapper around 1 or more containers
Who knows it	Docker, the runtime	Kubernetes
IP address	Uses the pod's IP	One IP for all its containers
Usually	One per pod	One container inside



Kubernetes runs **pods**, not containers. Containers in a pod start, run and die together, on the same node.

Deployment

Building blocks 2 of 5

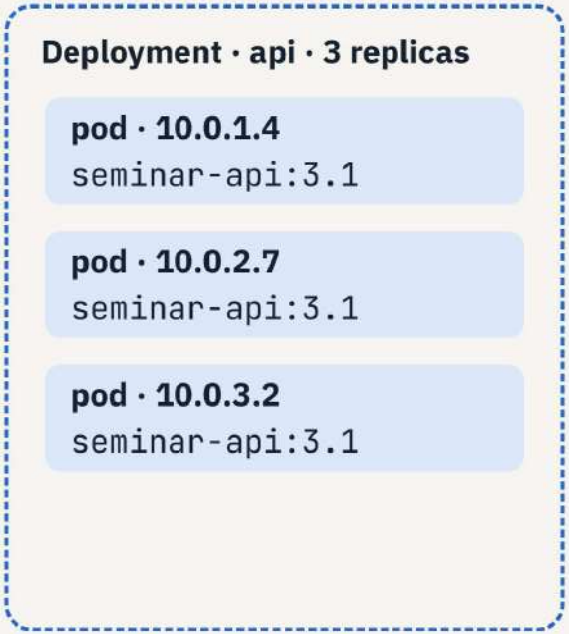
Keeps **N identical pods** running, and updates them.

A pod dies: it creates a new one.

New image: it replaces pods one by one.

Bad version: it rolls back to the previous one.

```yaml
kind: Deployment
metadata: { name: api }
spec:
  replicas: 3
  template: # the pod from before
```



Service

Building blocks 3 of 5

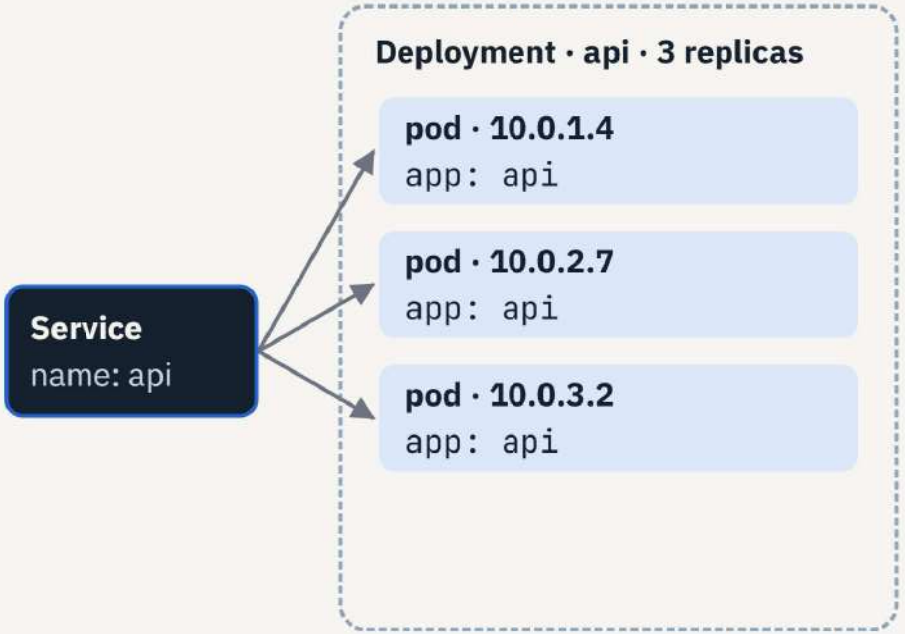
A **stable name and IP** in front of pods that come and go.

Finds its pods by label: **app: api**.

Spreads requests over the ready pods.

Other apps call **http://api**, never a pod IP.

```yaml
kind: Service
metadata: { name: api }
spec:
  selector: { app: api }
  ports: [{ port: 8081, targetPort: 8080 }]
```



Ingress

Building blocks 4 of 5

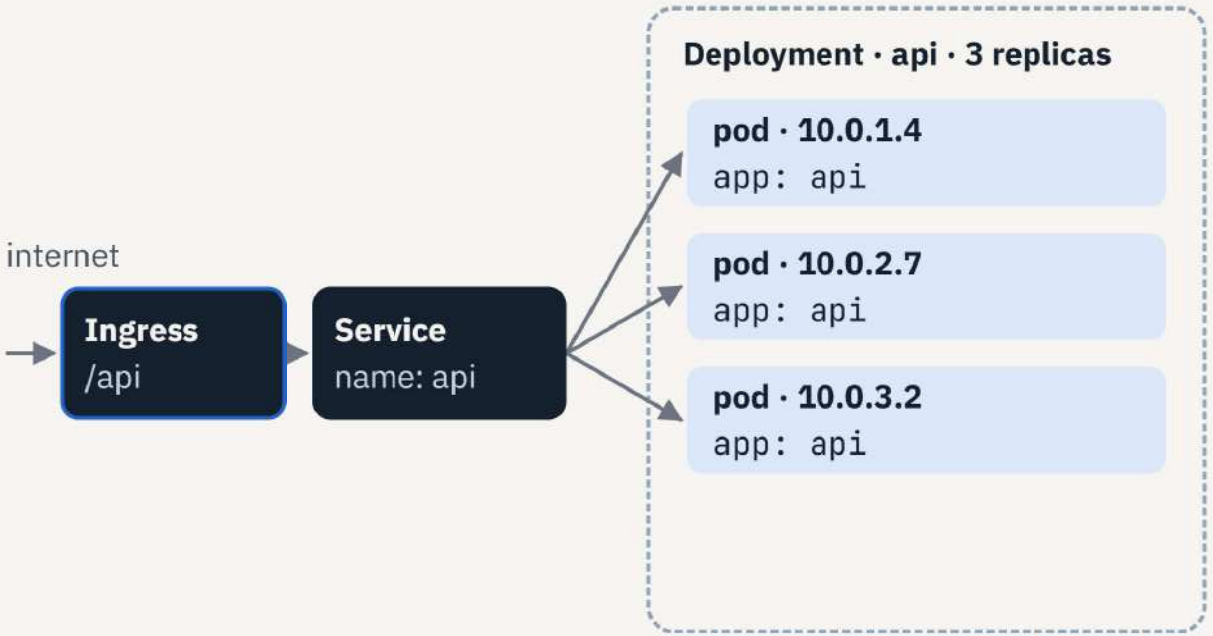
The **door from the internet**: HTTP routing by host or path.

/api goes to the api Service, **/** to the web Service.

One public IP and HTTPS for many services.

Needs an ingress controller, e.g. NGINX.

```
kind: Ingress
spec:
  rules:
    - http:
        paths:
          - path: /api
            backend: { service: { name: api
} }
```



HPA: Horizontal Pod Autoscaler

Building blocks 5 of 5

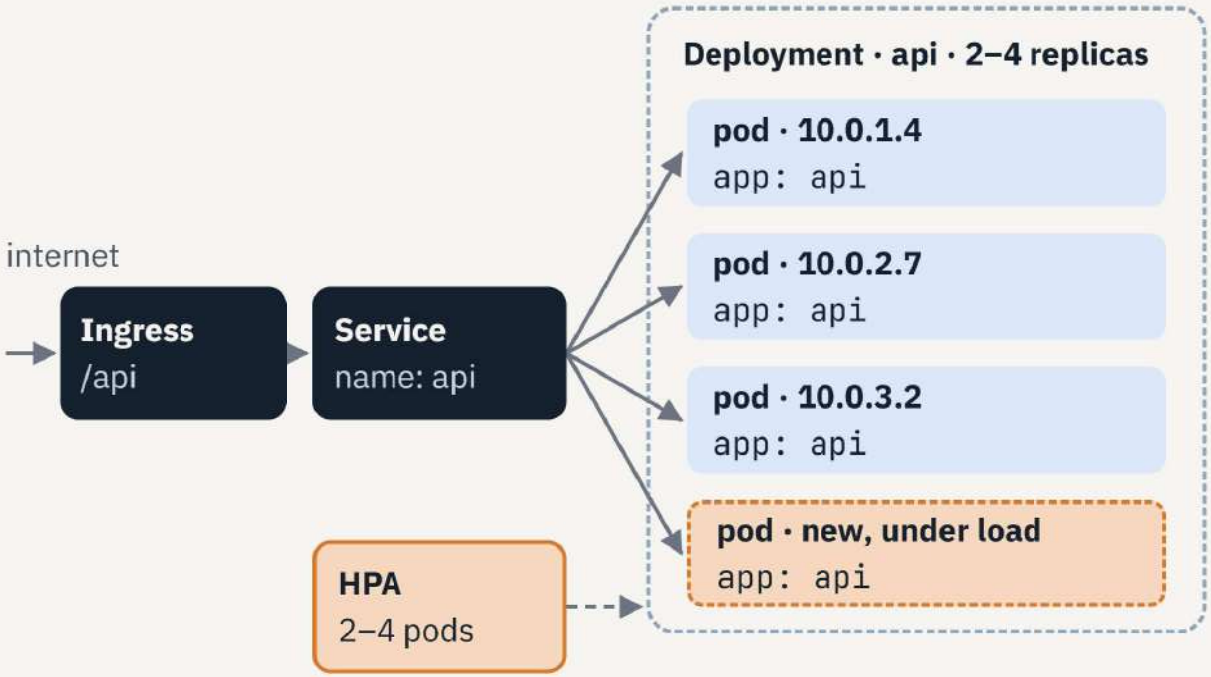
Adds pods when they are busy, **removes them when it is quiet**.

Ours: 2 to 4 pods, target 20% CPU.

Measured: **2 → 4** under load, back to 2 after.

Horizontal = more copies, not a bigger machine.

```
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { name: api }
  minReplicas: 2
  maxReplicas: 4
  # target: 20% CPU
```



The full picture

Ingress: the door from the internet

Service: a stable name in front of the pods

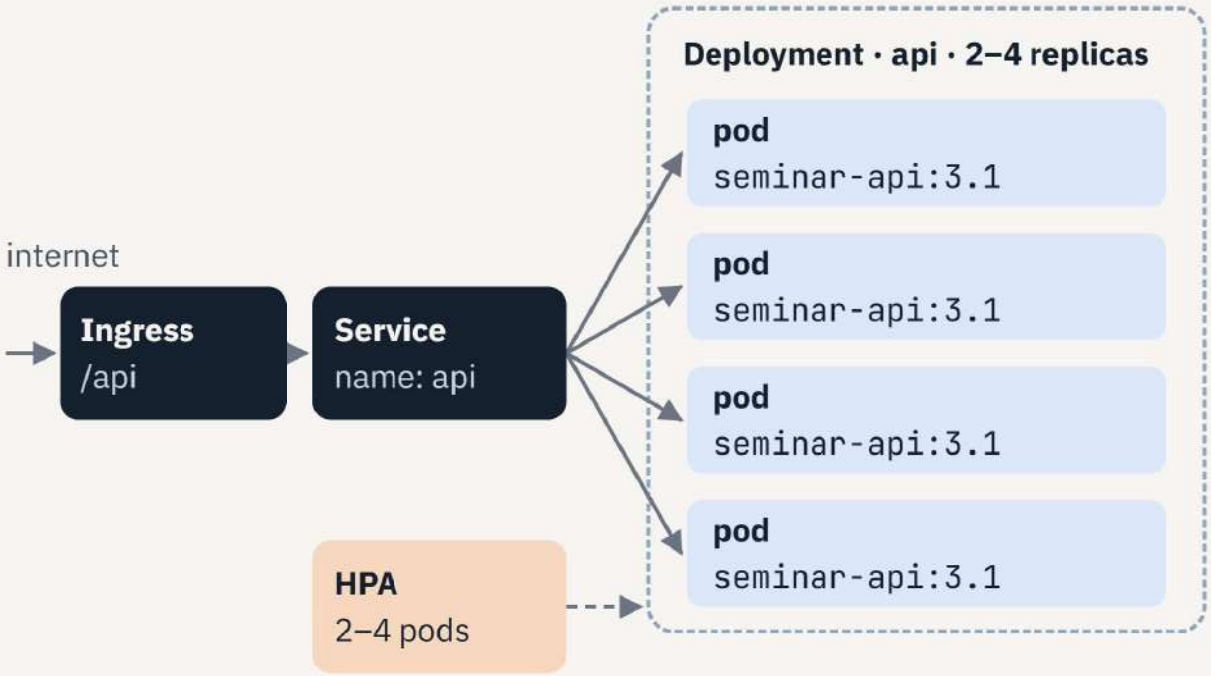
Deployment: keeps N pods running and updates them

Pod: one running container, disposable

HPA: changes N with the load

ConfigMap / Secret: settings and passwords for the pods. Secrets are only base64, not encrypted.

Node / cluster: the machines all of this runs on.



Do you need Kubernetes?

IT PAYS OFF WITH

Many services and several teams

Complex scaling or networking

The need to move between clouds

People to run and upgrade it

SOMETHING SIMPLER WITH

A small team, a few services

Nobody whose job is the cluster

A tight budget

→ Container Apps, App Service, Cloud Run

Our case: one API, so **Container Apps**. Kubernetes (AKS) is the bonus.

QUIZ · BLOCK 3A · 1 OF 2

What happens to data inside a container when its pod restarts?

A It is kept

B It is lost, unless it’s on a volume or in a managed database

C Kubernetes backs it up automatically

D It moves to the new pod

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 3A · 1 OF 2

What happens to data inside a container when its pod restarts?

A It is kept

B It is lost, unless it’s on a volume or in a managed database ✓

C Kubernetes backs it up automatically

D It moves to the new pod

Why: A container’s writable layer is deleted with it. Data that must survive lives on a volume, or better, in a managed database.

QUIZ · BLOCK 3A · 2 OF 2

Pods come and go with new IPs. What gives them one stable address?

A A Deployment

B A node

C A Service

D An HPA

Answer in the chat: **A, B, C or D**

ANSWER · BLOCK 3A · 2 OF 2

Pods come and go with new IPs. What gives them one stable address?

A A Deployment

B A node

C **A Service** ✓

D An HPA

Why: A Service gives one fixed name and IP, and sends traffic to the ready pods that carry its label.

HOMEWORK FOR TONIGHT

Before tomorrow's session

REQUIRED

Finish Hands-on 1

Your **api** image builds and runs. Tomorrow you push it to the cloud.

Behind? git switch step-1-containerized

OPTIONAL · 5 MIN

Try Compose yourself

On **step-2-compose-db**, follow README section 3, "Run everything".

Add a note, then down and down -v: is it still there? Why?

DAY 2 · FRIDAY 30 OCTOBER 2026

In the cloud

Yesterday on your laptop. Today: deploy it, automate it, run it.

code ▷ container ▷ cloud ▷ automation ▷ operations

David Cediel

Yesterday in one slide

Landscape

IaaS → CaaS → PaaS → FaaS: you do less, control less

Containers

An image packs the app and everything it needs. Config from env vars.

Compose

Many containers, one file. Data lives in volumes, not containers.

Kubernetes

You declare, it keeps it true. Pod, Deployment, Service, Ingress, HPA.

Today: **your image from yesterday goes to the cloud**, then we automate it and look at production.

SETUP CHECK · 2 MIN

Ready for the cloud?

1 · YOUR IMAGE

The api image builds and runs (yesterday's homework)

2 · DOCKER HUB

An account, and you know your username

3 · AZURE

portal.azure.com open, Azure for Students active

No image? Use the ready one in Hands-on 2: davidcediel12/seminar-api:1.0